

Reporting regression results

Extracting model estimates and data structure

Daniela Palleschi

2024-07-09

Table of contents

Set-up	2
Packages	2
Data	3
Data dictionary	3
Model prep	3
.	4
.	4
1 Linear regression	5
1.1 Naming conventions	5
1.2 Model summary	6
1.3 Extracting information	6
1.4 Model reporting	7
2 Running models reproducibly	8
3 Model reporting	8
3.1 broom	8
3.2 Publication-ready tables	8
4 Mixed models	9
4.1 lmer()	9
4.2 summary()	9
4.3 broom.mixed	10
4.3.1 tidy()	10
4.4 lmerTest	11
4.4.1 Publication-ready tables	11

5	How to report	12
5.1	Data analysis	12
5.2	Results	12
5.3	Example model report	12
5.4	Output	13
6	Bayesian models	13
6.1	brms	13
6.2	brm()	13
6.3	Saving our model	14
6.4	Mixed models in brm()	14
7	Plotting our brms model	16
8	Under the hood: brms	18
8.1	Understanding our model	18
9	Citing resources	19
	Session Info	21

Learning objectives

Today we will...

- run our first linear (mixed) model
- produce model summary tables
- report random effects
- do the same with a Bayesian model

Resources

Set-up

Packages

- we'll need the following packages for today

```
pacman::p_load(
  tidyverse,
  lme4,
  broom,
  broom.mixed,
  brms
)
```

Data

- we'll need the dataset we've been working with

```
df_lifetime <- readr::read_csv(here::here("data", "tidy_data_lifetime_pilot.csv"),
                              # for special characters
                              locale = readr::locale(encoding = "latin1")
                              ) |>
mutate_if(is.character, as.factor) |> # all character variables as factor
filter(type == "critical", # only critical trials
       px != "px3") # this participant had lots of 0's for some reason
```

Data dictionary

- let's review the data dictionary

```
dict_lifetime <- read_csv(
  here::here("data", "tidy_data_lifetime_pilot_dictionary.csv")
)
```

Model prep

- we want to only have critical items

```
df_lifetime <-
df_lifetime |>
filter(type == "critical") |>
droplevels()
```

- we want our predictors (`lifetime` and `tense`) to be factors

```
df_lifetime$lifetime <- as.factor(df_lifetime$lifetime)
df_lifetime$tense <- as.factor(df_lifetime$tense)
```

- and to have sum contrast coding:

- PP and `living` are -0,5
- SF and `dead` are +0,5

```
contrasts(df_lifetime$lifetime)
```

```
      living
dead      0
living    1
```

```
contrasts(df_lifetime$tense)
```

```
      SF
PP     0
SF     1
```

- we see the default treatment (or ‘dummy’) contrasts here
- to change them:

```
contrasts(df_lifetime$lifetime) <- c(-0.5, +0.5)
contrasts(df_lifetime$tense) <- c(+0.5, -0.5)
```

```
contrasts(df_lifetime$lifetime)
```

```
      [,1]
dead  -0.5
living 0.5
```

```
contrasts(df_lifetime$tense)
```

```
 [,1]  
PP  0.5  
SF -0.5
```

1 Linear regression

-

```
lm(  
  log(fp) ~ tense * lifetime,  
  subset = region == "verb" & ff > 0,  
  data = df_lifetime  
)
```

Call:

```
lm(formula = log(fp) ~ tense * lifetime, data = df_lifetime,  
    subset = region == "verb" & ff > 0)
```

Coefficients:

(Intercept)	tense1	lifetime1	tense1:lifetime1
5.63477	0.03357	-0.10130	-0.08320

1.1 Naming conventions

- we want to store our model as an object in the Environment
 - always try to use a prefix that defines what the object is
 - e.g., `fit_`, `mod_`, `lm_`, `glm_`
 - whatever you choose, be consistent within your project

```
lm_fp <-  
  lm(  
    log(fp) ~ tense * lifetime,  
    subset = region == "verb" & ff > 0,  
    data = df_lifetime  
  )
```

1.2 Model summary

- the most straightforward way to inspect your model and the results is with `summary()`

```
summary(lm_fp)
```

Call:

```
lm(formula = log(fp) ~ tense * lifetime, data = df_lifetime,  
    subset = region == "verb" & ff > 0)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-1.18141	-0.34282	0.00058	0.26923	1.38822

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.63477	0.01877	300.193	< 2e-16 ***
tense1	0.03357	0.03754	0.894	0.37158
lifetime1	-0.10130	0.03754	-2.698	0.00718 **
tense1:lifetime1	-0.08320	0.07508	-1.108	0.26828

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4374 on 539 degrees of freedom

Multiple R-squared: 0.01712, Adjusted R-squared: 0.01165

F-statistic: 3.129 on 3 and 539 DF, p-value: 0.02538

1.3 Extracting information

- `lme4` objects contains lots of information
 - navigate to the model object in your Environment and explore the elements
- `nobs(model)` will tell you how many data points were included in the model
- `formula(model)` will give you the model formula

```
nobs(lm_fp)
```

```
[1] 543
```

```
formula(lm_fp)
```

```
log(fp) ~ tense * lifetime
```

- want to check which contrasts were used in your model?

```
lm_fp$contrasts
```

```
$tense
```

```
[,1]
```

```
PP 0.5
```

```
SF -0.5
```

```
$lifetime
```

```
[,1]
```

```
dead -0.5
```

```
living 0.5
```

1.4 Model reporting

- when you run your models, you'll want to also print the output
 - this way you avoid needing to re-run your code to see the results
- for lme4 models we can use `summary()`
- number of participants in your model

```
summary(lm_fp)$ngrps["px"]
```

```
NULL
```

2 Running models reproducibly

- as your models get more computational complex, they can take time to fit
 - sometimes you also run several models in the same script
- with literate programming, we want to be able to produce dynamic reports
 - we don't want to run our models anew every time we re-render our document!
 - this also might result in different results if e.g., we update a package or R version
 - or if we send the script to somebody else with a different set-up
- ideally, we would have our source code, output (results), *and* the actual model all stored and retrievable

3 Model reporting

- when you run your models, you'll want to also print the output
 - this way you avoid needing to re-run your code to see the results
- for `lme4` models we can use `summary()`

3.1 broom

- the `broom` package also contains useful functions for reporting `lme4` models
 - `broom::tidy()` produces the model estimates for our fixed effects

3.2 Publication-ready tables

- we've already seen how to produce publication-ready tables
 - TASK: produce a summary of the fixed effects of a model and feed it into our `knitr` and `kableExtra` functions to produce a publication-ready table

4 Mixed models

- the assumption of independence is often violated
 - due to multiple observations collected from e.g., the same person or item, language, etc.
- this can lead to the inflation of Type I error (the chance of a false positive)
- to account for this nonindependence of data points, we can use *mixed* models
 - *mixed* because they contain fixed effects (i.e., our predictors at the population-level) as well as *random* effects (group-level overall means and predictor effects)

4.1 lmer()

```
lmer_lifetime <-  
  lmer(log(fp) ~ tense * lifetime +  
        (1|px) + (1|item_id),  
        subset = region == "verb" & fp > 0,  
        data = df_lifetime)
```

4.2 summary()

```
summary(lmer_lifetime)
```

Linear mixed model fit by REML ['lmerMod']

Formula: log(fp) ~ tense * lifetime + (1 | px) + (1 | item_id)

Data: df_lifetime

Subset: region == "verb" & fp > 0

REML criterion at convergence: 582.5

Scaled residuals:

	Min	1Q	Median	3Q	Max
	-2.99792	-0.67159	0.02895	0.67995	2.67790

Random effects:

Groups	Name	Variance	Std.Dev.
item_id	(Intercept)	0.007012	0.08374

```

px      (Intercept) 0.034124 0.18473
Residual              0.154885 0.39355
Number of obs: 543, groups:  item_id, 80; px, 7

```

Fixed effects:

	Estimate	Std. Error	t value
(Intercept)	5.63161	0.07245	77.736
tense1	0.03794	0.03389	1.120
lifetime1	-0.10308	0.03387	-3.044
tense1:lifetime1	-0.09587	0.06778	-1.414

Correlation of Fixed Effects:

	(Intr)	tense1	lftm1
tense1	-0.001		
lifetime1	0.000	0.013	
tense1:lftm1	0.003	0.002	-0.002

4.3 broom.mixed

- the `broom.mixed` package also contains useful functions for reporting `lme4` mixed models
 - `broom.mixed::tidy()` produces the model estimates for fixed and random effects
 - if you only want fixed effects, use the argument `effects = "fixed"`
 - if you only want random effects, use the argument `effects = "ran_pars"`

4.3.1 tidy()

```
tidy_lmer_lifetime <- broom.mixed::tidy(lmer_lifetime)
```

```
tidy_lmer_lifetime
```

```

# A tibble: 7 x 6
  effect   group term          estimate std.error statistic
<chr>   <chr>   <chr>          <dbl>    <dbl>    <dbl>
1 fixed   <NA>      (Intercept)    5.63     0.0724    77.7
2 fixed   <NA>      tense1         0.0379    0.0339     1.12
3 fixed   <NA>      lifetime1     -0.103    0.0339    -3.04
4 fixed   <NA>      tense1:lifetime1 -0.0959    0.0678    -1.41
5 ran_pars item_id sd__(Intercept)  0.0837    NA         NA
6 ran_pars px      sd__(Intercept)  0.185     NA         NA
7 ran_pars Residual sd__Observation  0.394     NA         NA

```

4.4 lmerTest

- is something missing from our model summary?
 - which effects were statistically significant ($p < .05$)?
- calculating p -values is not trivial
 - lme4 doesn't do it for linear models
- we can use the lmerTest package, which also has lme4

```
lmerTest_lifetime <-  
  lmerTest::lmer(log(fp) ~ tense * lifetime +  
    (1|px) + (1|item_id),  
    subset = region == "verb" & fp > 0,  
    data = df_lifetime)
```

```
tidy_lmerTest_lifetime <- broom.mixed::tidy(lmerTest_lifetime)
```

```
tidy_lmerTest_lifetime
```

```
# A tibble: 7 x 8  
  effect    group    term          estimate std.error statistic    df    p.value  
  <chr>    <chr>    <chr>          <dbl>    <dbl>    <dbl> <dbl>    <dbl>  
1 fixed    <NA>    (Intercept)    5.63     0.0724    77.7    6.20  1.65e-10  
2 fixed    <NA>    tense1         0.0379    0.0339     1.12  471.   2.63e- 1  
3 fixed    <NA>    lifetime1     -0.103    0.0339    -3.04  468.   2.47e- 3  
4 fixed    <NA>    tense1:lifeti~ -0.0959    0.0678    -1.41  471.   1.58e- 1  
5 ran_pars item_id  sd__(Intercep~ 0.0837    NA        NA      NA    NA  
6 ran_pars px      sd__(Intercep~ 0.185     NA        NA      NA    NA  
7 ran_pars Residual sd__Observati~ 0.394     NA        NA      NA    NA
```

4.4.1 Publication-ready tables

- we've already seen how to produce publication-ready tables
 - TASK: produce publication-ready tables using knitr and kableExtra for:
 - * a table of the model estimates of the fixed effects of our model
 - * a table of the model estimates of the random effects of our model
 - * a table of the with both the fixed and random effects
- produce one of these tables as LaTeX code (if you haven't already)
 - can you then produce this table in Overleaf?

5 How to report

- you would typically have a (sub)section called Data Analysis which includes all the steps taken prior to inspecting your results
 - i.e., how you prepared and analysed your data
 - this should be information stated in a pre-registration but in the past tense
- this would typically be followed by a Results section
 - containing the results from these cleaning/analysis steps

5.1 Data analysis

- should include, e.g.,
 - model summaries in tables and/or in-text
 - any inclusion/exclusion criteria for participants or observations
 - any transformations performed on dependent or independent variables
 - * e.g., we log-transformed first-pass reading times, and used sum contrast coding for our two 2-level predictors
 - * we should therefore define which levels were coded as what value (+/-0.5)
- model selection
 - what model did you start with? E.g., maximal model given the data and research questions (à la **barr_random_maximal_2013?**)
 - how did you handle convergence issues? E.g., parsimonious model selection (à la **bates_parsimonious_2015?**)

5.2 Results

- this would be followed by a section called Results, which reports e.g.,
 - the number of participants and trials ex-/included in analyses
 - possibly raw means and 95% confidence intervals
 - model formula
 - information about the number of observations per model
- a structure overview of model results

5.3 Example model report

```
An effect of *lifetime* was found in first-pass reading times at the verb region (Est = `r s
```

5.4 Output

An effect of *lifetime* was found in first-pass reading times at the verb region (Est = -0.1, $t = -3$, $p = 0.0025$). A significant effect of *tense* was not found found (Est = 0.038, $t = 1.1$, $p = 0.26$), nor an interaction of *lifetime* and *tense* (Est = -0.096, $t = -1.4$, $p = 0.16$).

6 Bayesian models

- this is an incredibly quick introduction to Bayesian linear regression
 - there is a lot of information that I’m glossing over
- the point here is to expose you to (potentially) your first Bayesian model
 - and to see how similar the input and output is to frequentist models

6.1 brms

- Stan is a language for producing Bayesian models
- brms is an R-user-friendly package to fit Bayesian models with Stan in the background
 - the function `brm()` can be used to fixed mixed models
- it doesn’t have the `subset` argument, so let’s first filter our data accordingly

```
df_lifetime_verb <-  
df_lifetime |>  
filter(region == "verb",  
       fp > 0)
```

6.2 brm()

```
brm_fp <-  
brm(fp ~ tense * lifetime,  
    family = lognormal(),  
    data = df_lifetime_verb)
```

6.3 Saving our model

- brms models have a nice argument where you can specify where to save a model directly:
file =
 - this is extremely useful as your models get more complex, because Bayesian models especially can take a long time to run

```
brm_fp <-  
brm(fp ~ tense * lifetime,  
    family = lognormal(),  
    data = df_lifetime_verb,  
    file = here::here("models", "brm_fp"))
```

6.4 Mixed models in brm()

- we can use the same function, just specify random effects

```
brm_fp <-  
brm(fp ~ tense * lifetime +  
    (1|item) + (1|px),  
    family = lognormal(),  
    data = df_lifetime_verb,  
    file = here::here("models", "brm_fp"))
```

```
summary(brm_fp)
```

```
Family: lognormal  
Links: mu = identity; sigma = identity  
Formula: fp ~ tense * lifetime  
Data: df_lifetime_verb (Number of observations: 543)  
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
       total post-warmup draws = 4000
```

Regression Coefficients:

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	5.63	0.02	5.60	5.67	1.00	4507	3386
tense1	0.03	0.04	-0.04	0.11	1.00	5252	3241
lifetime1	-0.10	0.04	-0.17	-0.03	1.00	4313	3173
tense1:lifetime1	-0.08	0.07	-0.23	0.07	1.00	5040	3227

Further Distributional Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	0.44	0.01	0.41	0.47	1.00	5471	3106

Draws were sampled using `sampling(NUTS)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat = 1`).

- the output is the same...why's that?
 - when we specify `file =`, `brms` will first look to see if such a file already exists
 - if it does, it simply loads in that model rather than fitting the model
 - the downside is that it doesn't check if the model formula has changed, so we might not be fitting the model at all!
- to work around this, either
 - delete the file with the same name, if you're *sure* you want to overwrite it
 - change the filename (which I suggest always matches the object name in your script)
- we'll do the latter

```
brm_fp_mixed <-  
brm(fp ~ tense * lifetime +  
    (1|item_id) + (1|px),  
    family = lognormal(),  
    data = df_lifetime_verb,  
    file = here::here("models", "brm_fp_mixed"))
```

```
summary(brm_fp_mixed)
```

```
Family: lognormal  
Links: mu = identity; sigma = identity  
Formula: fp ~ tense * lifetime + (1 | item_id) + (1 | px)  
Data: df_lifetime_verb (Number of observations: 543)  
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
       total post-warmup draws = 4000
```

Multilevel Hyperparameters:

~item_id (Number of levels: 80)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	0.07	0.03	0.01	0.13	1.00	1004	1132

~px (Number of levels: 7)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	0.24	0.10	0.12	0.49	1.00	972	1262

Regression Coefficients:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	5.63	0.10	5.42	5.83	1.00	1104	1329
tense1	0.04	0.03	-0.03	0.11	1.00	6820	2958
lifetime1	-0.10	0.03	-0.17	-0.04	1.00	7347	2785
tense1:lifetime1	-0.10	0.07	-0.23	0.04	1.00	8084	2718

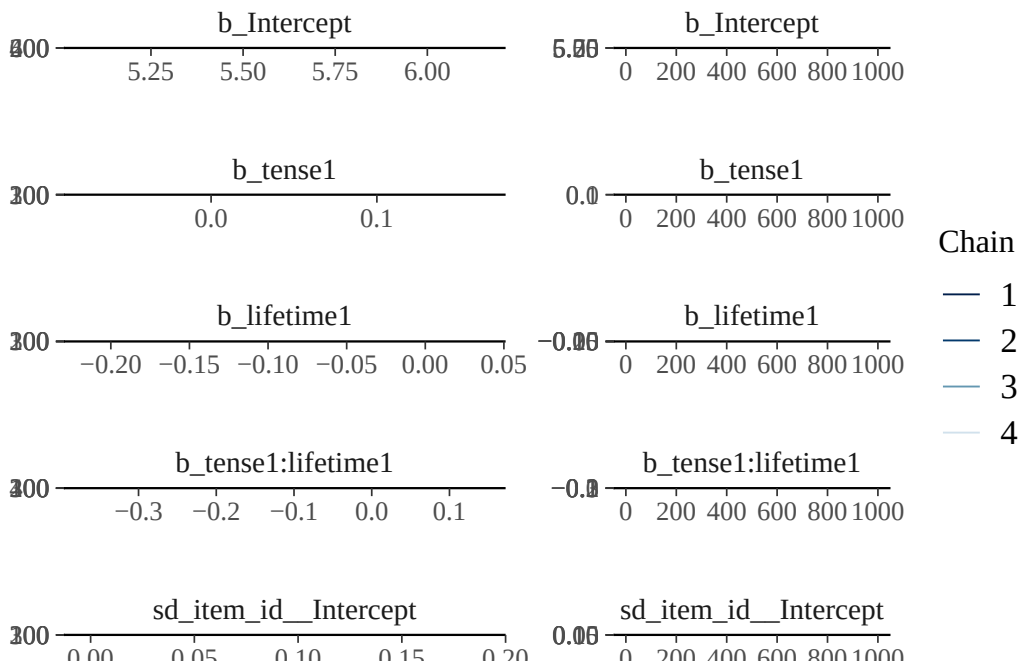
Further Distributional Parameters:

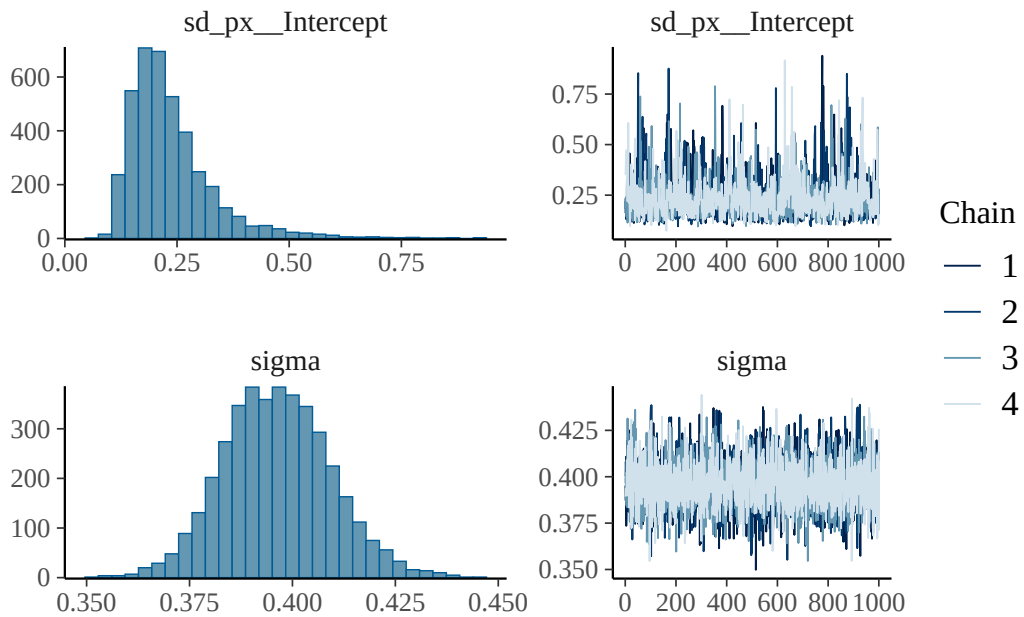
	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	0.40	0.01	0.37	0.42	1.00	3793	2636

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

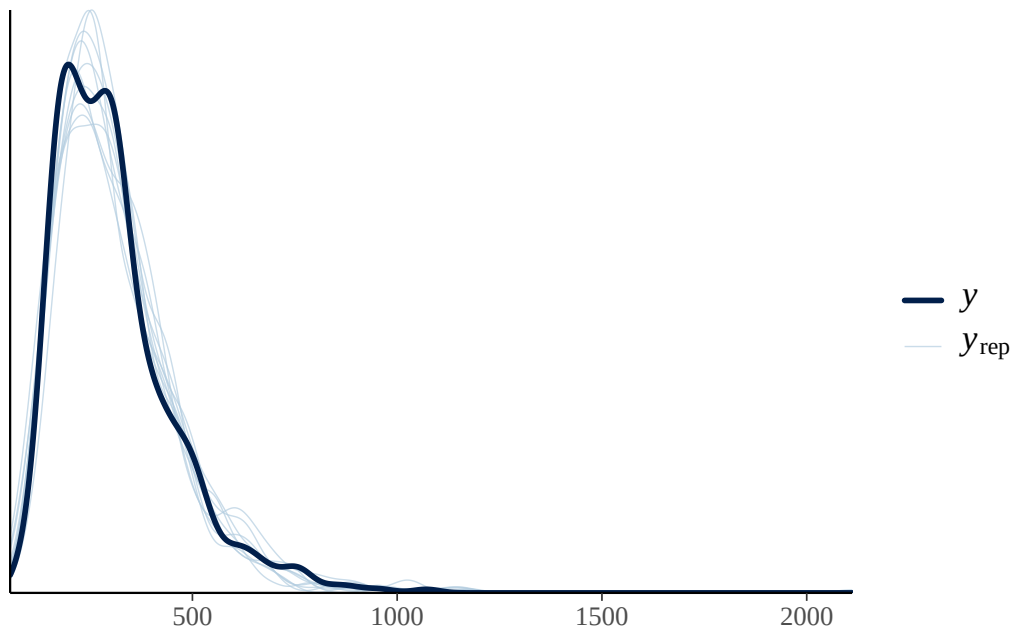
7 Plotting our brms model

```
plot(brm_fp_mixed)
```





```
pp_check(brm_fp_mixed)
```



- the thick dark line is the *posterior distribution*
 - i.e., what our model fit based on the data
- the thin blue lines represent 10 random draws/distributions from the model

8 Under the hood: brms

- we skipped over a lot of information, most notably
 - *priors*, which are the real bonus of Bayesian modelling
- Bayes' theorem states that the probability

8.1 Understanding our model

- some of the same functions apply to `brms` models

```
formula(brm_fp)
```

```
fp ~ tense * lifetime
```

```
nobs(brm_fp)
```

```
[1] 543
```

```
summary(brm_fp)
```

```
Family: lognormal
Links: mu = identity; sigma = identity
Formula: fp ~ tense * lifetime
Data: df_lifetime_verb (Number of observations: 543)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000

Regression Coefficients:
              Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept          5.63      0.02    5.60    5.67 1.00     4507     3386
tense1              0.03      0.04   -0.04    0.11 1.00     5252     3241
lifetime1          -0.10      0.04   -0.17   -0.03 1.00     4313     3173
tense1:lifetime1   -0.08      0.07   -0.23    0.07 1.00     5040     3227
```

Further Distributional Parameters:

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	0.44	0.01	0.41	0.47	1.00	5471	3106

Draws were sampled using `sampling(NUTS)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat = 1`).

9 Citing resources

- you should cite the packages you used to fit your models
 - as well as other package versions used (but `SessionInfo()` also takes care of this)
- to cite packages in-text, you can first extract the citation: with

```
citation(package = "brms")
```

To cite brms in publications use:

```
Paul-Christian Bürkner (2017). brms: An R Package for Bayesian  
Multilevel Models Using Stan. Journal of Statistical Software, 80(1),  
1-28. doi:10.18637/jss.v080.i01
```

```
Paul-Christian Bürkner (2018). Advanced Bayesian Multilevel Modeling  
with the R Package brms. The R Journal, 10(1), 395-411.  
doi:10.32614/RJ-2018-017
```

```
Paul-Christian Bürkner (2021). Bayesian Item Response Modeling in R  
with brms and Stan. Journal of Statistical Software, 100(5), 1-54.  
doi:10.18637/jss.v100.i05
```

To see these entries in BibTeX format, use `'print(<citation>, bibtex=TRUE)'`, `'toBibtex(.)'`, or set `'options(citation.bibtex.max=999)'`.

- or as BibTeX citation:

```
print(citation(package = "brms"), bibtex=TRUE)
```

To cite brms in publications use:

```
Paul-Christian Bürkner (2017). brms: An R Package for Bayesian  
Multilevel Models Using Stan. Journal of Statistical Software, 80(1),  
1-28. doi:10.18637/jss.v080.i01
```

A BibTeX entry for LaTeX users is

```
@Article{,  
  title = {{brms}: An {R} Package for {Bayesian} Multilevel Models Using {Stan}},
```

```

author = {Paul-Christian Bürkner},
journal = {Journal of Statistical Software},
year = {2017},
volume = {80},
number = {1},
pages = {1--28},
doi = {10.18637/jss.v080.i01},
encoding = {UTF-8},
}

```

Paul-Christian Bürkner (2018). Advanced Bayesian Multilevel Modeling with the R Package brms. The R Journal, 10(1), 395-411.
doi:10.32614/RJ-2018-017

A BibTeX entry for LaTeX users is

```

@Article{,
  title = {Advanced {Bayesian} Multilevel Modeling with the {R} Package {brms}},
  author = {Paul-Christian Bürkner},
  journal = {The R Journal},
  year = {2018},
  volume = {10},
  number = {1},
  pages = {395--411},
  doi = {10.32614/RJ-2018-017},
  encoding = {UTF-8},
}

```

Paul-Christian Bürkner (2021). Bayesian Item Response Modeling in R with brms and Stan. Journal of Statistical Software, 100(5), 1-54.
doi:10.18637/jss.v100.i05

A BibTeX entry for LaTeX users is

```

@Article{,
  title = {Bayesian Item Response Modeling in {R} with {brms} and {Stan}},
  author = {Paul-Christian Bürkner},
  journal = {Journal of Statistical Software},
  year = {2021},
  volume = {100},
  number = {5},
  pages = {1--54},
  doi = {10.18637/jss.v100.i05},
}

```

```
encoding = {UTF-8},  
}
```

- then copy it to your `.bib` file, add a citation reference key (e.g., `burkner_bayesian_2021`), and cite it in-text with `@burkner_bayesian_2021`, which will be formatted as Bürkner (2021)
 - alternatively, save the package reference in Zotero and access it using `rbbt`
 - e.g., copy the DOI then find the Zotero button for ‘Add Item by Identifier’

Learning objectives

Today we...

- ran our first linear (mixed) model
- produced model summary tables
- reported random effects
- did the same with a Bayesian model

Session Info

```
print(sessionInfo(), locale = F)
```

```
R version 4.4.0 (2024-04-24)  
Platform: aarch64-apple-darwin20  
Running under: macOS Ventura 13.2.1
```

```
Matrix products: default
```

```
BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib  
LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices datasets  utils      methods    base
```

```
other attached packages:
```

```
[1] brms_2.21.5      Rcpp_1.0.12      broom.mixed_0.2.9.5  
[4] broom_1.0.6      lme4_1.1-35.3    Matrix_1.7-0  
[7] lubridate_1.9.3  forcats_1.0.0    stringr_1.5.1  
[10] dplyr_1.1.4      purrr_1.0.2      readr_2.1.5
```

```
[13] tidyr_1.3.1          tibble_3.2.1          ggplot2_3.5.1
[16] tidyverse_2.0.0
```

loaded via a namespace (and not attached):

```
[1] gridExtra_2.3          inline_0.3.19          rlang_1.1.4
[4] magrittr_2.0.3         frrrr_0.3.1           matrixStats_1.3.0
[7] compiler_4.4.0         loo_2.7.0             vctrs_0.6.5
[10] reshape2_1.4.4         pkgconfig_2.0.3       crayon_1.5.2
[13] fastmap_1.2.0          backports_1.5.0       labeling_0.4.3
[16] utf8_1.2.4             rmarkdown_2.27        tzdb_0.4.0
[19] nloptr_2.0.3           tinytex_0.51          bit_4.0.5
[22] xfun_0.44              jsonlite_1.8.8        parallel_4.4.0
[25] R6_2.5.1               stringi_1.8.4         StanHeaders_2.32.9
[28] parallelly_1.37.1      boot_1.3-30           numDeriv_2016.8-1.1
[31] estimability_1.5.1     rstan_2.32.6          knitr_1.47
[34] pacman_0.5.1           bayesplot_1.11.1      splines_4.4.0
[37] timechange_0.3.0       tidysselect_1.2.1     rstudioapi_0.16.0
[40] abind_1.4-5            yaml_2.3.8            codetools_0.2-20
[43] curl_5.2.1             listenv_0.9.1         pkgbuild_1.4.4
[46] lattice_0.22-6         lmerTest_3.1-3        plyr_1.8.9
[49] withr_3.0.0            bridgesampling_1.1-2  posterior_1.5.0
[52] coda_0.19-4.1          evaluate_0.23         future_1.33.2
[55] RcppParallel_5.1.7     pillar_1.9.0          tensorA_0.36.2.1
[58] checkmate_2.3.1        renv_1.0.7            stats4_4.4.0
[61] distributional_0.4.0    generics_0.1.3        vroom_1.6.5
[64] rprojroot_2.0.4        hms_1.1.3             rstantools_2.4.0
[67] munsell_0.5.1          scales_1.3.0          minqa_1.2.7
[70] globals_0.16.3         xtable_1.8-4          glue_1.7.0
[73] emmeans_1.10.2         tools_4.4.0           mvtnorm_1.2-5
[76] grid_4.4.0             QuickJSR_1.2.2        colorspace_2.1-0
[79] nlme_3.1-165           cli_3.6.2             fansi_1.0.6
[82] Brodningnag_1.2-9      V8_4.4.2              gtable_0.3.5
[85] digest_0.6.35          farver_2.1.2          htmltools_0.5.8.1
[88] lifecycle_1.0.4        here_1.0.1            bit64_4.0.5
[91] MASS_7.3-60.2
```

References

Bürkner, P.-C. (2021). Bayesian Item Response Modeling in *R* with **Brms** and *Stan*. *Journal of Statistical Software*, 100(5). <https://doi.org/10.18637/jss.v100.i05>